

Q1-In the following you will find the description of a company which has different departments with employees. **All fields must be private, and all methods listed for each question must be public. Any auxiliary helper method you create must be private. You may assume that all arguments used in method calls have valid values, so you do not need to use defensive programming (check for validity of arguments)**

1-Implement a class called **Employee**. The class has the following fields: **name** of type **String**, **departmentId** of type **int**, which represents the department where the employee works, **salary** of type **int**, **assignedTasks** of type **List<String>**, which represents the list of tasks that are assigned to the employee to process, and **totalProcessed** of type **int**, that represents the number of processed tasks.

2-Implement the constructor **Employee(String name, int salary, int departmentId)** that initialises the fields with the value of the corresponding parameters. Furthermore, it initialises **totalProcessed** to zero and **assignedTasks** with an empty list.

3-Implement methods **getTotalProcessed()**, **getDepartmentId()**, and **getSalary()** that return the value of **totalProcessed**, **departmentId**, and **salary**, respectively, and implement the method **getAssignedTasksNum()** that returns the size of the **assignedTasks** list.

4-Implement a method **setSalary(int newSalary)** that sets the value of **salary** to **newSalary**.

5-Implement a method **processTask()** that prints

"processing task <t>" where <t> is the task at index 0 in **assignedTasks**, if **assignedTasks** is non-empty, removes this task from the list, and increases the value of **totalProcessed** by one.

"no tasks to process" otherwise.

6-Implement a method **toString()**, which returns a string "**name: <name>, department Id: <departmentId>, salary: <salary>**", where <x> is the value of field **x**.

7-Implement a method **addTask(String newTask)**, that adds **newTask** to the end of the list **assignedTasks**.

8-Implement a class called **Company**. The class has a field called **allEmployees** of type **Set<Employee>**. In the constructor of the class initialise **allEmployees** with an empty set.

9-In the **Company** class, implement a method **addEmployee(Employee emp, int salary)** such that it sets the salary of **emp** to **salary** and adds the employee **emp** to **allEmployees**.

10-In the **Company** class, implement a method **findLeastBusy(int depId)** which returns the employee in **allEmployees** who has the smallest number of assigned tasks (size of the **assignedTasks** list) and whose **departmentId** is **depId**. Assume that there is at least one employee in each department. If there are multiple employees with the smallest number of assigned tasks, return any of them.

11-In the **Company** class, implement a method called **assignTask(String task, int depId)** that adds **task** to the **assignedTasks** list of an employee in **allEmployees** who has smallest number of assigned tasks in department **depId**. (Hint, use **findLeastBusy** method as a helper method)

12-In the **Company** class, implement a method called **raiseSalary(int percentage)** which finds the employee(s) in **allEmployees** that have the highest number of processed tasks, increases the salary of these employees by **percentage** percent, using **Math.round()**, and prints the information of these employees according to the format explained in task 6. For example, if percentage is 20, the salary is increased by 20 percent for all relevant employees. If there are multiple employees with highest number of processed tasks, the printing order does not matter.

Note: To make this work you will need to use casting between the primitive types as **Math.round** expects values of either type **float** or **double** to function, and the result can be either a value of type **int** or **long** and, if it is the latter, it must be cast to an **int**.

Q2-Consider the code below. Which of the following option(s) can be used for creating an object of type **Room**?

```
public class Room{  
    public Room(int area, String[] names){...}  
    public Room(){...}  
}
```

- a) new Room()
- b) new Room(100, new String[]{"Per","Ton","Siph"})
- c) new Room(200, {"4A05"})
- d) new Room(200)

Q3-Select the correct statement(s).

- a) A class can extend multiple classes
- b) An interface can implement multiple interfaces
- c) High coupling is desirable in class design
- d) High cohesion is desirable in class design

Q4-Assuming that **x** and **y** are of type **int**, what is the result of printing "**x**" + **y** + "**1**" if value of **x** is 10 and value of **y** is 7?

- a) 108
- b) 171
- c) x71
- d) x8

Q5-Consider the code below and that the **Robot** class has only one constructor. Which of the following option(s) is/are correct about **MiniRobot**?

```

public class Robot{
    private String model;
    public Robot (String model){
        this.model = model;
    }
    ...
}

```

```

public class MiniRobot extends Robot{...}

```

- a) All constructors of **MiniRobot** must start with a call to the constructor of **Robot**
- b) The class **MiniRobot** can have at most one constructor
- c) All constructors of class **MiniRobot** should receive at least one parameter
- d) **MiniRobot** is a subclass of **Robot**

Q6-Consider the code below. Select the correct option(s).

```

public interface InfoProc{
    public String sendInfo();
    public void printInfo();
    public String decodeInfo(String info);
}

```

- a) Any non-abstract class implementing **InfoProc** has at least three methods
- b) A class implementing **InfoProc** cannot have a method with header **String printInfo()**
- c) Any class implementing **InfoProc** must have a constructor that receives one parameter
- d) An object of type **InfoProc** can be created using **new InfoProc()**

Q7-Which of the following should replace (*) in the code such that all the even numbers are removed from **myList**.

```

public void removeEven(List<Integer> myList){
    Iterator<Integer> itr = myList.iterator();
    while( itr.hasNext() ){
        (*)
    }
}

```

- a) if(itr.next() / 2 == 0) itr.remove();
- b) if(itr.next() % 2 == 0) myList.remove(itr.next());
- c) if(itr.next() % 2 == 0) itr.remove();
- d) if(itr.next() % 2 != 0) myList.remove(itr.next());

Q8-Consider the code below and that it runs successfully. Select option(s) that show a correct mapping of keys and values. (Assume **x->y** means key **x** is mapped to value **y**)

```
var myMap = new HashMap<String, Integer>();
myMap.put("Hana",12);
myMap.put("Tam", 24);
myMap.put("Phil",40);
myMap.put("Chila",32);
myMap.remove("Tam");
myMap.put("Phil",41);

var x= myMap.get("Hana")-2;
```

- a) "Hana"->10
- b) "Phil"->40
- c) "Chila"->32
- d) "Phil"->41
- e) "Hana"->12
- f) "Tam"->24

Q9-Consider the code below. Select option(s) which are suitable for testing the output of the **check** method. (Assume that the code shows the intended functionality.)

```
public class C {
    public static boolean check(int x, int[] myArray){
        boolean flag = false;
        for(var item: myArray){
            if(x>item){
                flag = true;
            }
        }
        return flag;
    }
}
```

- a) assertEquals(false, C.check(5, new int[]{1,2,3,4,6}))
- b) assertEquals(false, C.check(1, new int[]{5,1,2,10}))
- c) assertTrue(C.check(-10, new int[]{10,-3,-2,20}))
- d) assertEquals(false,C.check(0, new int[10]))

Q10-Which of the following option(s) can be a correct replacement for (*).

```
public (*) myMethod(List<Integer> discounts){  
    int x = discounts.size()-1;  
    if (x>0)  
        return discounts.get(x)+"%";  
    else  
        return discounts.size()+"";  
}
```

- a) int
- b) String
- c) void
- d) Integer

Q11-Which of the following is/are primitive type(s) in Java?

- a) char
- b) Square
- c) float
- d) String
- e) Integer
- f) long

Q12-Assume that **arr** is of type **int[]** and that the following code runs successfully. What is the value of **arr[5]** after executing the code below?

```
for(int i = 10;i>0; i--){  
    arr[i] = i % 3;  
}
```

- a) 3
- b) 2
- c) 5
- d) 6

Q13-Consider the code below. What is the number of loop iterations?

```
int x= 30;  
while (x>0){  
    x=x/2;  
}
```

- a) 30
- b) 5
- c) 4
- d) 6
- e) 29

Q14-Assume that **weights** is of type **ArrayList<Double>** and is empty before running the code below. Given that the code runs successfully, what value is returned by the method call in the last line?

```
weights.add(10.3);  
weights.add(54.1);  
weights.add(85.2);  
weights.add(67.9);  
double x = weights.remove(2)+1;  
  
weights.add(41.9);  
weights.remove(2);  
weights.get(2);
```

- a) 68.9
- b) 41.9
- c) 54.1
- d) 85.2

Q15-Consider the code below. Which of the option(s) is/are correct after executing the code.

```
String[] a = new String[]{"A","C","G","K"};
```

```
String[] b = new String[10];
```

```
b = a;
```

```
a[1] = "H";
```

- a) length is 10
- b) length is 4
- c) b[1] is equal to "H"
- d) a[3] is equal to "G"

Q16-Consider the class definition below.

```
public class Proc{  
    private static int count;  
    public Proc(){count++;}  
    public void setCount(int newCount){count = newCount;}  
}
```

Assuming that the following lines of code are executed successfully. What is the value of the field **count** for objects **p1**, **p2**, **p3** in order?

```
Proc p1 = new Proc();  
Proc p2 = new Proc();  
Proc p3 = new Proc();  
p1.setCount(8);  
new Proc();
```

- a) 8,2,3
- b) 9,8,8
- c) 8,8,8
- d) 9,9,9